Building an AI-Assisted Remediation Agent for Ansible Automation Platform Workflows

Author: Manus AI

Date: May 25, 2026

Executive summary

You can build this as an **event-driven remediation service** that listens for failed Ansible Automation Platform, or **AAP**, workflow/job events, collects detailed failure context from the AAP controller API, classifies the failure, proposes or applies a safe fix in GitHub or Azure DevOps, and then retries the automation under strict guardrails. The most important design principle is that the agent should not blindly mutate production automation. It should separate **safe transient retry**, **code remediation through a pull request**, and **human-escalation-only failures**.

AAP already provides the core integration points you need. Automation controller supports notification templates, including webhooks, and those notifications can be associated with job templates and workflows at trigger levels such as started, success, and error. [1] AAP workflows are graph-like compositions of job templates and related nodes, and the workflow job tracks the jobs spawned by that graph. [2] The controller API exposes `/api/v2/` resources, including relaunch operations, stdout, job events, workflow jobs, and related workflow job nodes. [3] For repository-side remediation, GitHub provides REST endpoints for pull requests and repository content operations, [4] [5] while Azure DevOps provides Git push and pull request APIs with code-write scope. [6] [7]

> **Recommended starting point:** Build a small Python service with a webhook endpoint, a background worker, a database table for incidents, an AAP API client, a GitHub/Azure DevOps repository adapter, and a policy engine. Start with "diagnose and open PR," then add automatic retry only for low-risk transient failures.

Architecture options

Before selecting an implementation, decide how much you want the system to own. A lightweight implementation can diagnose and create tickets or pull requests, while a more autonomous implementation can patch, validate, merge, and rerun. In production, I would recommend a **human-approved remediation loop** for code changes and an **automatic retry loop** only for known transient failures.

Approach	Tradeoffs	Cost	Setup complexity
Webhook listener plus manual approval	AAP sends failed-job notifications to your service. The service diagnoses, creates a pull request or incident, and waits for human approval before retrying. This is the safest option and the best first implementation.	Low infrastructure cost; mainly a small always-on service and optional LLM usage.	Moderate; requires AAP notification configuration, API tokens, repository credentials, and a small database.
Fully autonomous remediation agent	The agent diagnoses, patches code, validates, merges, and relaunches. This can reduce toil but introduces risk if policy, tests, or permissions are weak.	Higher LLM and validation cost; still modest infrastructure cost.	High; requires strong guardrails, approvals, rollback, audit, and environment-specific testing.
Polling-only monitor	The service polls AAP for failed jobs instead of receiving webhooks. This works when inbound webhooks are difficult, but it is slower and less efficient.	Low to moderate depending on polling frequency.	Low to moderate; easier network setup, but requires careful rate limits and state tracking.
ChatOps-assisted remediation	The agent posts findings to Slack, Teams, or email and waits for an operator command such as “approve retry” or “open PR.” This is operationally friendly and auditable.	Low to moderate.	Moderate; requires chat integration and identity mapping.

Recommended end-to-end design

The best production design is a **closed-loop remediation pipeline** with strong boundaries between observation, diagnosis, code change, and retry. AAP should remain the system of record for workflow execution. GitHub or Azure DevOps should remain the system of record for automation code. The remediation service should maintain an audit database that records incidents, logs, classifications, patch attempts, pull requests, approvals, retry attempts, and final outcomes.

Component	Responsibility	Notes
AAP notification webhook	Sends failed workflow/job events to the remediation service.	AAP webhook notification templates are suitable because AAP can send notifications when jobs fail. [1]
AAP API client	Fetches workflow/job metadata, related nodes, stdout, events, failed hosts, extra variables, and source revision.	Use a service account with the minimum permissions needed to view and relaunch relevant templates.
Incident database	Stores deduplication keys, status, classification, retry count, PR links, and audit trail.	Use PostgreSQL, MySQL, or SQLite for an MVP.
Failure classifier	Distinguishes transient errors, code defects, inventory issues, credentials, permissions, and platform capacity failures.	Use deterministic rules first, then LLM summarization for ambiguous logs.
Repository adapter	Reads and updates code from GitHub or Azure DevOps.	Abstract this behind a provider interface so the rest of the agent is repository-platform agnostic.
AI remediation engine	Proposes minimal patches based on failure context and repository contents.	Constrain it to the repo, require patch diffs, and validate before any retry.
Validation runner	Runs syntax checks, linting, unit tests, and optionally a dry-run job template.	For Ansible, use `ansible-playbook --syntax-check`, `ansible-lint`, `yamllint`, and targeted molecule tests where available.
Approval and policy engine	Decides whether to auto-retry, open a PR, request approval, or escalate.	Never auto-change credentials, inventory, RBAC, secrets, or production host limits.
Retry controller	Relaunches the failed workflow/job or launches the	Use AAP relaunch endpoints where appropriate; otherwise

	template again with equivalent variables.	relaunch the workflow template with controlled variables. 3
Notification layer	Posts results to Slack, Teams, email, Jira, ServiceNow, or GitHub/Azure DevOps comments.	Include diagnosis, evidence, PR link, retry job ID, and final state.

Core workflow

The operational flow should be intentionally conservative. The service receives a failed AAP notification, validates that the request came from AAP, creates or updates an incident record, enriches the incident with AAP API data, classifies the failure, and chooses one of several remediation paths.

Failure class	Example signal	Recommended action
Transient infrastructure failure	Temporary package repository outage, connection reset, intermittent DNS, cloud API rate limit, lock timeout.	Retry automatically with exponential backoff, capped retry count, and no code change.
Ansible syntax or module error	YAML parse failure, undefined variable, invalid module parameter, missing collection.	Create a branch and pull request with a minimal fix; retry only after checks and approval.
Inventory or host reachability issue	`unreachable`, SSH timeout, WinRM timeout, stale host, wrong limit.	Open an incident and optionally notify inventory owners; do not patch playbook code unless the inventory source is in Git and the fix is obvious.
Credential or permission failure	Authentication failed, forbidden, sudo password missing, vault secret unavailable.	Escalate to owner; do not let the agent create or alter secrets automatically.
Platform or capacity failure	AAP execution node down, insufficient capacity, image pull failure, controller issue.	Create an operations incident; optionally retry after platform health recovers.
Potentially destructive task failure	Storage, firewall, identity, production database, or	Require human approval for any remediation and retry.

	security policy changes.	

MVP implementation plan

For a first version, build the smallest reliable loop: webhook ingestion, AAP context retrieval, classification, incident creation, PR creation for likely code issues, and manual retry approval. This gives you useful automation without creating a risky self-modifying system.

Phase	Deliverable	Success criteria
Phase 1: Observe	A webhook endpoint receives AAP failure notifications and stores an incident.	A failed workflow creates exactly one deduplicated incident.
Phase 2: Enrich	The service fetches AAP job details, stdout, job events, workflow nodes, project revision, and extra variables.	The incident page or log contains enough context for a human to diagnose the failure.
Phase 3: Classify	Rules identify transient, code, inventory, credential, and platform failures.	At least 70–80% of common known failures are classified without LLM intervention.
Phase 4: Propose	The agent creates a GitHub or Azure DevOps branch and PR for code-like failures.	The PR contains a minimal patch, explanation, links to the failed job, and validation results.
Phase 5: Retry	Approved remediation triggers AAP relaunch with retry limits and audit logging.	Retry job IDs and outcomes are recorded and sent to operators.
Phase 6: Harden	Add RBAC, policy-as-code, rate limits, rollback, dashboards, and ChatOps approvals.	The system is safe enough for selected production workflows.

AAP configuration

In AAP, configure notification templates for the workflow job templates or job templates you want to monitor. The official documentation states that users create notification templates through `/api/v2/notification_templates`, then associate them with job templates or other supported objects at the appropriate trigger level, such as error. [1]

> Red Hat describes the notification flow as creating a notification template, assigning it to supported objects, and choosing trigger levels such as started, success, or error. [1]

For this use case, configure a **Webhook notification template** that sends JSON to your remediation service. Use an internal HTTPS endpoint if possible. Include a shared secret header such as `X-AAP-Webhook-Secret`, and ensure your service validates the secret, source network, and payload shape before doing any work.

AAP item	Recommended setting
Notification type	Webhook
Trigger	Error/failure, and optionally started/success for correlation
Scope	Start with one non-production workflow template, then expand by organization/project
Payload destination	`https://remediator.example.com/webhooks/aap`
Auth	Static secret header, mTLS, API gateway signature, or private network access
Service account	Read job/workflow details; relaunch only approved templates

Repository strategy: GitHub and Azure DevOps

Treat code changes as normal software changes. The agent should create a branch, commit a patch, and open a pull request rather than pushing directly to a protected branch. GitHub's REST API supports pull request operations, and GitHub's repository contents API supports creating and modifying Base64-encoded repository content. [4] [5] Azure DevOps exposes APIs to push changes to a Git repository and create pull requests. [6] [7]

Capability	GitHub	Azure DevOps
Read files	Repository Contents API or `git clone` with token. [5]	Git Items API or `git clone` with PAT/OAuth.
Create branch and commit	Git Data API, Contents API, or `git` CLI with token. [5]	Pushes API or `git` CLI with PAT/OAuth. [7]
Open pull request	Pulls API. [4]	Pull Requests Create API. [6]

Required permission	Fine-grained token or app permissions for contents and pull requests.	`vso.code_write` or equivalent service connection scope. [6] [7]
Best practice	Use a GitHub App for enterprise deployments.	Use an Azure DevOps service principal/OAuth app where possible; PAT only for MVP.

Python service skeleton

The following example shows the shape of a small FastAPI service. In production, move long-running remediation work to a queue such as Celery, RQ, Dramatiq, Sidekiq, or a cloud-native queue. The webhook handler should verify, persist, and enqueue quickly rather than holding the HTTP request open.

Python

```python
# app.py
import hmac
import os
from fastapi import FastAPI, Header, HTTPException, Request, BackgroundTasks
from pydantic import BaseModel

from aap_client import AAPClient
from remediation import handle_failed_aap_event

APP_SECRET = os.environ["AAP_WEBHOOK_SECRET"]

app = FastAPI(title="AAP Remediation Agent" )

class AAPWebhookEvent(BaseModel):
    id: int | None = None
    name: str | None = None
    status: str | None = None
    url: str | None = None
    unified_job_id: int | None = None
    workflow_job_id: int | None = None
    job: int | None = None


def verify_secret(value: str | None) -> None:
    if not value or not hmac.compare_digest(value, APP_SECRET):
        raise HTTPException(status_code=401, detail="Invalid webhook secret")
```

```python
@app.post("/webhooks/aap")
async def aap_webhook(
    request: Request,
    background_tasks: BackgroundTasks,
    x_aap_webhook_secret: str | None = Header(default=None),
):
    verify_secret(x_aap_webhook_secret)
    payload = await request.json()

    # Store the raw payload and create a deduplication key before processing.
    # Example key: f"aap:{payload.get('unified_job_id') or payload.get('id')}:{
    background_tasks.add_task(handle_failed_aap_event, payload)
    return {"accepted": True}
```

AAP API client skeleton

The exact endpoint details can differ by controller version and job type, but the general pattern is stable: call the controller `/api/v2/` resources using a service token, retrieve job/workflow details, obtain stdout and events, and relaunch through the relevant relaunch or template-launch endpoint. The Red Hat API catalog lists relaunch endpoints such as `POST /api/v2/workflow_jobs/{id}/relaunch/` and many related job resources. 3

Python

```python
# aap_client.py
import os
import requests

class AAPClient:
    def __init__(self):
        self.base_url = os.environ["AAP_BASE_URL"].rstrip("/")
        self.session = requests.Session()
        self.session.headers.update({
            "Authorization": f"Bearer {os.environ['AAP_TOKEN']}",
            "Accept": "application/json",
        })
        self.verify_tls = os.getenv("AAP_VERIFY_TLS", "true").lower() == "true"

    def get(self, path: str, **params):
        response = self.session.get(
            f"{self.base_url}{path}", params=params, verify=self.verify_tls, ti
        )
        response.raise_for_status()
        return response.json()

    def post(self, path: str, payload: dict | None = None):
```

```
        response = self.session.post(
            f"{self.base_url}{path}", json=payload or {}, verify=self.verify_tl
        )
        response.raise_for_status()
        return response.json() if response.content else {}

    def get_job(self, job_id: int):
        return self.get(f"/api/v2/jobs/{job_id}/")

    def get_job_stdout(self, job_id: int):
        response = self.session.get(
            f"{self.base_url}/api/v2/jobs/{job_id}/stdout/?format=txt",
            verify=self.verify_tls,
            timeout=60,
        )
        response.raise_for_status()
        return response.text

    def get_job_events(self, job_id: int):
        return self.get(f"/api/v2/jobs/{job_id}/job_events/", page_size=200)

    def relaunch_job(self, job_id: int):
        return self.post(f"/api/v2/jobs/{job_id}/relaunch/")

    def relaunch_workflow_job(self, workflow_job_id: int):
        return self.post(f"/api/v2/workflow_jobs/{workflow_job_id}/relaunch/")
```

Failure classification logic

The classifier should combine deterministic rules, historical outcomes, and LLM-based summarization. Do not start with an unconstrained AI agent. Start with a rule engine that maps known signals to action classes, and use the LLM to explain, rank likely causes, and propose small diffs only when the rule engine says it is safe.

Python

```
# classifier.py
from dataclasses import dataclass

@dataclass
class Classification:
    category: str
    confidence: float
    action: str
    reason: str
```

```
TRANSIENT_PATTERNS = [
    "connection reset by peer",
    "temporary failure in name resolution",
    "timed out",
    "rate limit",
    "failed to download metadata",
    "could not resolve host",
]

CREDENTIAL_PATTERNS = [
    "permission denied",
    "authentication failed",
    "invalid token",
    "forbidden",
    "sudo: a password is required",
]

CODE_PATTERNS = [
    "syntax error",
    "mapping values are not allowed",
    "undefined variable",
    "unsupported parameter",
    "couldn't resolve module/action",
]


def classify_failure(stdout: str, event_json: dict) -> Classification:
    text = stdout.lower()

    if any(p in text for p in TRANSIENT_PATTERNS):
        return Classification("transient", 0.80, "auto_retry", "Matches known t

    if any(p in text for p in CREDENTIAL_PATTERNS):
        return Classification("credential_or_permission", 0.85, "escalate", "Cr

    if any(p in text for p in CODE_PATTERNS):
        return Classification("code_defect", 0.75, "open_pr", "Likely repositor

    return Classification("unknown", 0.40, "request_human_review", "No safe det
```

Repository adapter pattern

Use an interface so GitHub and Azure DevOps behave the same to the rest of the agent. For an MVP, using `git clone`, a short-lived branch, and provider APIs for PR creation is often simpler than editing files through REST content APIs. For highly controlled environments, use provider-native APIs and avoid giving shell access to the agent.

Python

```python
# repo_adapter.py
from abc import ABC, abstractmethod
from pathlib import Path

class RepoAdapter(ABC):
    @abstractmethod
    def clone(self, dest: Path, ref: str | None = None) -> None: ...

    @abstractmethod
    def create_branch(self, branch_name: str) -> None: ...

    @abstractmethod
    def commit_all(self, message: str) -> str: ...

    @abstractmethod
    def push_branch(self, branch_name: str) -> None: ...

    @abstractmethod
    def open_pull_request(self, branch_name: str, title: str, body: str) -> str
```

The pull request body should include the failed AAP job URL, workflow ID, classification, evidence from the logs, changed files, validation commands, validation results, and retry policy. This makes the agent's work reviewable rather than mysterious.

Validation and guardrails

The validation stage is where most of the risk is reduced. Every code remediation should run local checks before a pull request is opened or updated. If the repository includes Molecule, unit tests, or integration tests, run those too. If it does not, add a first baseline of syntax and lint checks.

Guardrail	Purpose	Example implementation
Allowlist templates	Limit automation to selected workflows.	Only process workflow/job template IDs in `ALLOWED_TEMPLATE_IDS`.
Retry budget	Prevent loops.	Maximum two automatic retries per incident and one code-remediation retry per PR.
Protected branches	Prevent direct production mutation.	Require PRs against `main`, `master`, or release branches.

Sensitive failure blocklist	Avoid risky autonomous changes.	Block auto-remediation for credentials, secrets, RBAC, firewall, IAM, and destructive storage tasks.
Patch size limit	Prevent broad rewrites.	Reject patches changing more than N files or M lines unless approved.
Static validation	Catch obvious defects.	Run `ansible-playbook --syntax-check`, `ansible-lint`, `yamllint`, and repo tests.
Human approval	Keep production control with operators.	Require PR review or ChatOps approval before relaunch after code changes.
Audit log	Explain every action.	Store prompt, model, evidence, diff, validation, approval, and retry result.

Retry policy

Relaunching should be treated as a controlled operation. AAP exposes relaunch-style endpoints in the controller API, including workflow job relaunch operations in the API catalog.[3] Use relaunch when it preserves the right context. If relaunch is not available or not appropriate for a specific job type, launch the workflow job template again with the same approved survey variables and `extra_vars`.

Scenario	Retry decision
Transient failure with high confidence	Auto-retry after 2–5 minutes, then exponential backoff, with a maximum retry count.
Code fix PR opened but not merged	Do not retry the production workflow. Optionally run a non-production validation job.
Code fix merged	Sync project in AAP if required, then relaunch the workflow or launch the template again.
Credential or permission failure	Do not retry automatically. Retry only after an owner updates the credential or access.

Unknown failure	Require human approval and capture additional context.

AI remediation prompt pattern

The prompt to the AI coding component should be narrow and evidence-based. It should ask for a minimal patch, a reason, and tests. It should explicitly forbid changes outside the repository, secrets, credentials, and unrelated refactoring.

Plain Text

```
You are remediating an Ansible automation failure. Use only the provided reposi

Goal:
- Identify the smallest likely code change that fixes the failure.
- Do not modify secrets, credentials, inventory ownership, RBAC, or production
- Do not perform broad refactoring.
- Return a unified diff and a concise explanation.

Failure evidence:
- AAP workflow job: {workflow_job_url}
- Failed job: {job_url}
- Status: {status}
- Failed task/event excerpts: {event_excerpt}
- Stdout excerpt: {stdout_excerpt}
- Repository revision used by AAP: {scm_revision}

Repository context:
{selected_files}

Validation commands to satisfy:
- ansible-playbook --syntax-check {playbook}
- ansible-lint
- yamllint .
```

Deployment recommendation

Because this system must be available when AAP fails, it should run as an **always-on service** rather than as a one-off script. Good deployment targets include an internal VM, OpenShift/Kubernetes, Azure Container Apps, Azure App Service, AWS ECS/Fargate, or another persistent server environment. If your AAP controller is private, deploy the

remediation service inside the same network or expose it through an API gateway with mTLS and IP allowlisting.

Deployment target	When to use it
Internal VM or persistent Linux server	Best for a quick MVP, full control, private network access, and simple systemd deployment.
OpenShift/Kubernetes	Best if AAP and your platform teams already operate container workloads.
Azure Container Apps or App Service	Best if your enterprise standardizes on Azure and can connect privately to AAP.
Serverless function	Useful for ingestion only, but less ideal for long-running remediation and repository validation.

Minimal production data model

The database should be boring and auditable. You only need a few core tables at first.

Table	Purpose
`incidents`	One row per failed workflow/job, with dedupe key, AAP IDs, status, classification, and owner.
`incident_events`	Append-only timeline of webhook received, enrichment completed, PR opened, retry launched, and final result.
`remediation_attempts`	Proposed patch, model metadata, changed files, validation result, PR URL, and approval status.
`retry_attempts`	Relaunch endpoint used, new AAP job/workflow ID, retry number, result, and timestamps.
`policy_decisions`	Rule that allowed or blocked an action, with reason and actor.

Security model

Use separate service accounts and least privilege. The AAP token should be able to read job details and relaunch only the templates you permit. The repository token should be able to create branches and pull requests but should not be allowed to bypass branch protections. The LLM should receive only the minimum required logs and code snippets; redact secrets and environment-specific credentials from stdout before sending content to any model.

Secret	Recommended handling
AAP API token	Store in a secret manager, rotate regularly, use least privilege.
GitHub credential	Prefer GitHub App installation tokens over long-lived PATs.
Azure DevOps credential	Prefer OAuth/service principal where possible; use PAT only for MVP and rotate it.
Webhook secret	Store server-side only, validate on every request, rotate if exposed.
LLM API key	Restrict egress, log token usage, and redact sensitive content before prompts.

Practical build sequence

Start by proving the loop in non-production. Configure one AAP workflow template to send error webhooks to your service. Build the service to store the payload, call AAP for stdout and job events, and post a diagnosis to Slack, Teams, or email. Then add repository read access and implement PR creation for a narrow set of safe error types, such as YAML syntax failures or missing collection references. Finally, add controlled retry after approval.

Week	Build focus	Outcome
1	Webhook ingestion, incident storage, AAP API enrichment.	Failed AAP workflows produce searchable incident records.
2	Classifier, log summarization, notification.	Operators receive useful root-cause summaries.
3	GitHub/Azure DevOps repository adapter and PR creation.	Code-like failures produce reviewable pull requests.

4	Validation runner and approval workflow.	PRs include validation results and require approval.
5	Controlled relaunch and project sync.	Approved fixes trigger a new AAP run with audit trail.
6	Hardening, dashboards, policies, and runbooks.	The system is ready for selected production workflows.

What not to automate at first

Do not initially allow the agent to update secrets, credentials, privilege escalation settings, RBAC, production inventory membership, firewall rules, IAM policy, or destructive infrastructure tasks. Do not let it push directly to protected branches. Do not allow unlimited retries. Do not send unredacted stdout to an external LLM because Ansible output may contain environment details or accidental secrets.

Suggested first implementation decision

I would build the first version as a **Python FastAPI remediation service with a background worker**, deployed inside your network. It should use AAP webhook notifications for failures, AAP API calls for enrichment, deterministic classification for first-pass action selection, and provider-specific adapters for GitHub and Azure DevOps. It should create pull requests for code defects, automatically retry only known transient errors, and require approval before retrying anything changed by AI.

This design gives you the operational benefit you want—failed automation is detected, diagnosed, fixed, and rerun—without sacrificing auditability or change-control discipline.

References

[1] Red Hat Ansible Automation Platform 2.4: Notifications

[2] Red Hat Ansible Automation Platform 2.4: Workflows in automation controller

[3] Red Hat Developers: Ansible Automation Platform controller API

[4] GitHub REST API endpoints for pull requests

[5] GitHub REST API endpoints for repository contents

[6] Azure DevOps REST API: Pull Requests - Create

[7] Azure DevOps REST API: Pushes - Create